

TEC-PLUS 核心板

XC6SLX9-2FTG256 (核心板)

使用指南 V2.0.6

张改革 著

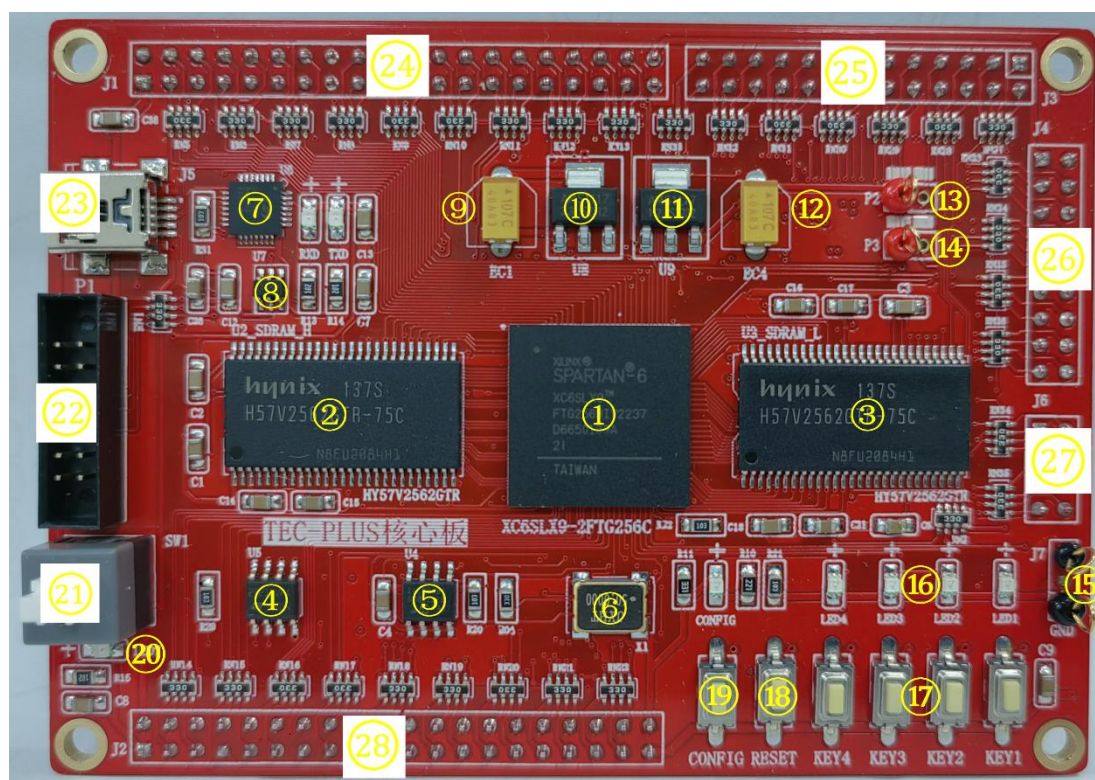
目录

第零部分：内容简介.....	1
TEC-PLUS 核心板概述.....	1
第一部分：软件的安装与使用.....	3
1.1 ISE14.7 软件安装.....	3
1.2 ModelsimSE 安装.....	5
1.3 ISE14.7 联合 ModelsimSE.....	6
1.4 CP2102 串口芯片驱动安装.....	8
1.5 ISE14.7 简介与教程.....	9
第二部分：板载资源简介与使用.....	22
2.1 按键监测或译码电路.....	22
2.2 流水灯.....	24
2.3 串口 Uart 通信.....	25
2.4 EPPROM 读写.....	27
2.5 FLASH 读写.....	28
2.6 SDRAM 读写.....	31
第三部分：大板与 FPGA 核心板连接信号汇总.....	32
01. 大板输入信号.....	32
02. 频率计实验.....	33
03. 蜂鸣器实验.....	34
04. 交通灯实验.....	35
05. VGA 实验.....	36
06. 控制器实验.....	37
第四部分：核心板 PIN 汇总.....	38

第零部分：内容简介

为了适应新阶段计算机专业，计算机专业课程的长足与可持续发展，经过长时间的精心设计，生产了 TEC-PLUS 设备，满足数字逻辑、计算机组成原理和计算机系统结构等三门课程的基础实验、教学和科研任务。

TEC-PLUS 核心板概述



TEC-PLUS 核心板基于 XILINX Spartan-6 (XC6SLX9-2FTG256) FPGA 器件，可以单独使用也可以配合底箱使用的一款的产品。实物图入上图展示所示。现将主要的元件简介如下：

- 标识 1: XILINX Spartan-6 XC6SLX9-2FTG256 FPGA 器件；
- 标识 2: 存储架构为 4Banks*4Mbits*16 同步动态 SDRAM 芯片 (HY57V2562 型号)；
- 标识 3: 存储架构为 4Banks*4Mbits*16 同步动态 SDRAM 芯片 (HY57V2562 型号)；
- 标识 4: 16Mbit SPI 接口的 FLASH 芯片 (ST 公司 M25P16)；
- 标识 5: 4Kbit IIC 接口的 EEPROM 芯片 (24LC04)；
- 标识 6: 50MHz 的时钟晶振；
- 标识 7: CP2102GM 芯片 (开发板和 PC 之间串口通信)；

- 标识 8: SRV05-4(低电容 ESD 静电保护二极管, 有效保护开发板);
- 标识 9:100uf/16V 胆电容;
- 标识 10:AMS1117-3.3V(输出电压为 3.3V 的正向低压降稳压器);
- 标识 11:AMS1117-1.2V(输出电压为 1.2V 的正向低压降稳压器);
- 标识 12:100uf/16V 胆电容;
- 标识 13:3.3V 电压测试端口;
- 标识 14:1.2V 电压测试端口;
- 标识 15:GND 测试端口;
- 标识 16:4 个独立的 LED 输出;
- 标识 17:4 个独立的按键开关;
- 标识 18:1 位复位按键开关,用于程序复位;
- 标识 19:1 位 CONFIG 按键, 用于 FPGA 的重新配置;
- 标识 20:LED 红灯, 独立用核心板时候查看电源供电;
- 标识 21:5V 电源开关, 独立情况下使用;
- 标识 22:核心板下载电缆使用的 JTAG 连接器, 用于对 FPGA 进行编程;
- 标识 23:MINI USB 接口, 和 CP2102 配合, 给开发板供电/通信;
- 标识 24:40 针扩展连接器 1(核心板丝印层标识 J1);
- 标识 25:26 针扩展连接器 2(核心板丝印层标识 J3);
- 标识 26:18 展连接器 3(核心板丝印层标识 J4);
- 标识 27:8 展连接器 4(核心板丝印层标识 J6);
- 标识 28:40 针扩展连接器 5(核心板丝印层标识 J2)

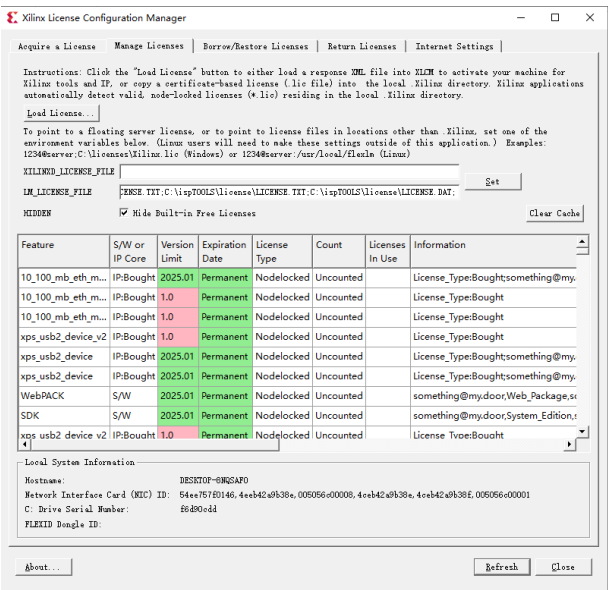
第一部分：软件的安装与使用

1.1 ISE14.7 软件安装

TEC-PLUS 核心板所用芯片是 Xilinx 公司的 Spartan-6 系列中的一款 FPGA 芯片，所以根据 Xilinx 所提供的编辑环境，大家需要先了解下 ISE14.7 这款软件，当然，现在 Xilinx 已经停止对 ISE14.7 进行更新，也就是说 ISE14.7 已经是最高版本，截止到现在我自己使用的是 WIN10 的系统，ISE14.7 还是可以使用的。WIN10 以上未测试，因为 ISE14.7 对 PC 的要求不是很高，所以现在一般性的 PC 都是可以满足的，如果软件安装过程出现问题，或者安装完成编译很慢请更新 PC。

因为软件牵扯到版权的问题，所以我个人不提供软件包，麻烦使用者辛苦一点去 Xilinx 官网或者百度网盘进行下载。下面直接介绍 ISE14.7 的安装方法。

下载完成以后，直接运行 Xilinx 安装包目录下的 xsetup.exe 应用程序。在弹出欢迎对话框的情况下和其他软件的安装一样选择 Next, 碰见需要同意的条款，只能选择同意，别无他法。最后出现 Install 选项，直接点击，开始安装。安装器件可能会跳出 MATLAB 安装的对话框，点击 OK 即可。需要说明的是，软件安装完成后还需要安装 License，不然无法进行编译。在弹出的 XILINX LICENSE Configuration Manager 里点击 Manage Licenses 按钮。



点击加载 License，在浏览框中选择软件安装目录中的破解文件 xilinx_ise.lic 文件。继续下一步，软件会提示安装成功，截止到这一步软件

就可以开始使用了。正所谓万物皆可仿真，所以咱们将仿真软件 Modelsim 进行仿真，下面将 Modelsim 软件一并安装，考虑到软件分 32 位和 64 位两种，用户可以安装自己 PC 的 OS 进行选择合适的版本。因为我用的是 Win10 工作站版，所以我选择 64 位的安装包。

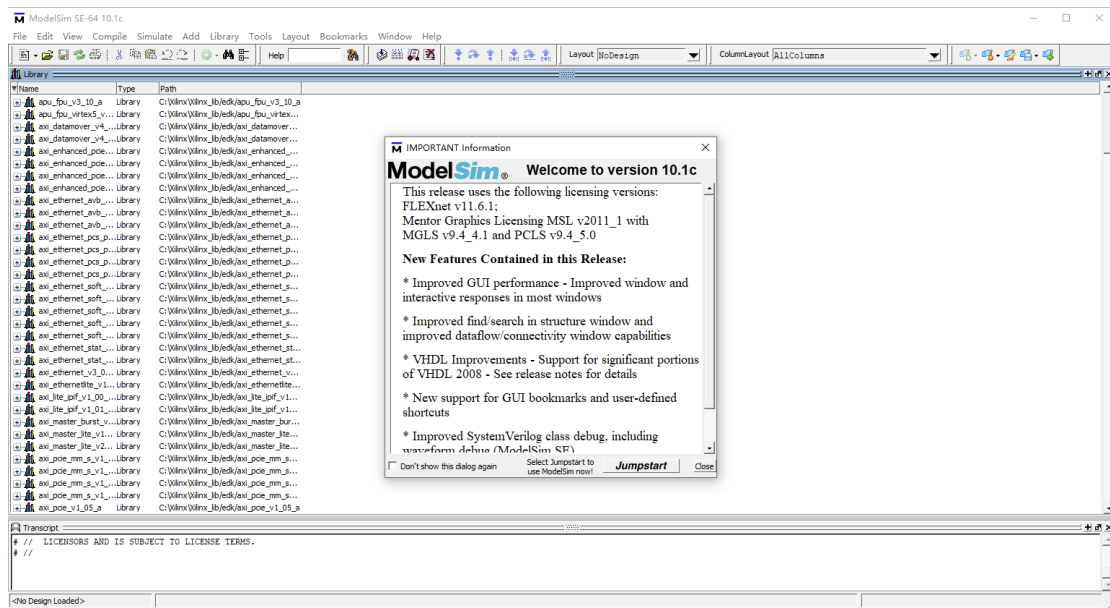


1.2 ModelsimSE 安装

找到 ModelsimSE 的安装目录，双击 .exe 文件，开始安装。出现安装界面后，点击 Next 继续，在随后出现的界面中，一般都使用默认选项，有 YES 的选择 YES，有 Agree 的选择 Agree，安装到最后，提示重新启动电脑即可。安装完成了，但是软件本身还需要破解，所以下面简单讲解下破解。

软件本身自带有 Crack 文件夹，复制目录下的两个文件，并且将其复制到软件的安装目录下，EX：C:\modeltech64_10.1c\win64。将 WIN64 文件中的 mgls64.dll 的文件属性中的只读去掉。在系统自带的附录目录下找到：命令提示符，已管理员身份运行，打开命令窗口。修改路径到 C:\modeltech64_10.1c\win64，在命令窗口输入：patch_dll.bat，回车，后续会生成 license，保存到安装目录。

添加系统环境变量，添加名称 LM_LICENSE_FILE, 变量值的路径与前面保存的 license 路径一致即可。截止到这一步，破解完成。



1.3 ISE14.7 联合 ModelsimSE

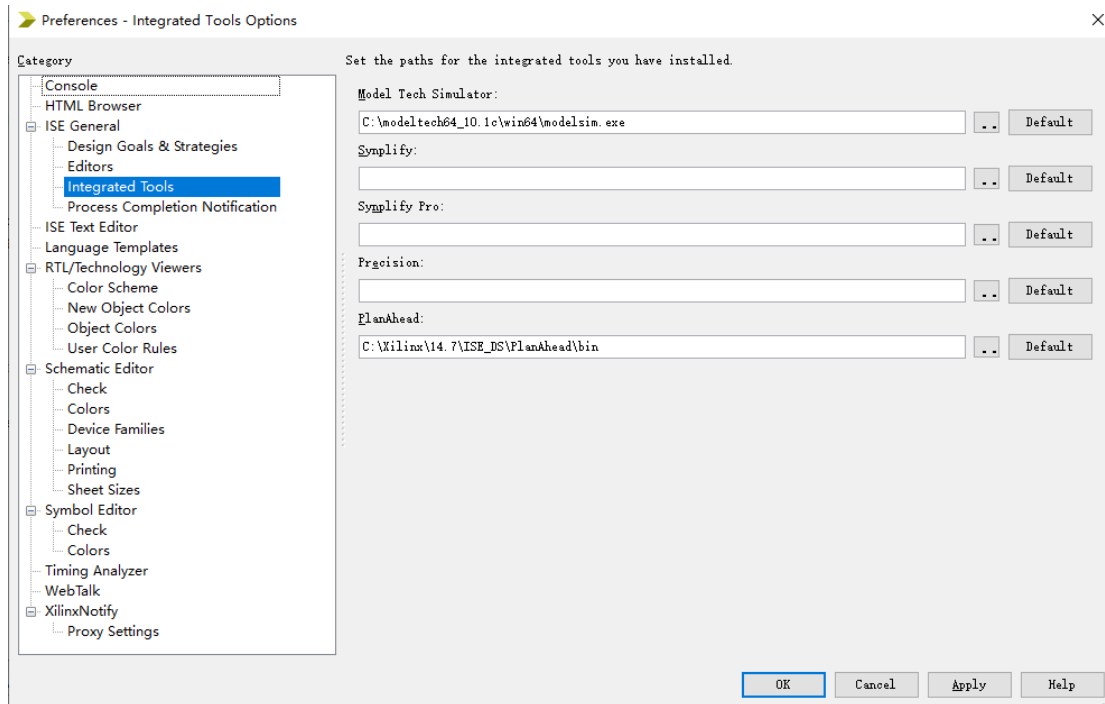
正所谓万物皆可仿真，仿真也是分老大老二的，ISE 软件在 EDA 设计方面是很强大的存在，但是在仿真方面就稍逊一筹，我个人比较喜欢 ModelSimSE 这款仿真软件，运行速度快，仿真性能好，查看各种信号和操作简单，关键是界面交互人性化。下面简单介绍 ISE 软件如何联合 ModelSimSE。

首先在开始菜单中，找到产生 ISE 仿真库文件。路径“Xilinx Design Tools”→“ISE Design Suite 14.7”→“ISE Design Tools”→“64-bit Tools”→“Simulation Library Complication Wizard”。

下面一步，选择自己已经安装好的 ModelSim 的版本，另外还要填入 Modesim.exe 所在的文件夹。特别注意模拟软件的安装路径。NEXT，选择需要编译的语言，VHDL 和 Verilog，当然也可以都选择，一般情况下都会选择两个都选择。方便后续开发。NEXT，需要编译的 Xilinx FPGA 和 CPLD 库。自己选择，不过默认全选，继续 NEXT-NEXT。填入输出已编译库的路径，默认是 C:\Xilinx\xilinx_lib。继续下一步：Launch Compiled Process。下面的时间一般来说比价长，主要是和自己 PC 的配置相关。耐心等待。最后会出现一个编译报告的 summary，点击 Finish 完成整个器件库的编译。

待库生成后，回到 ISE 的安装目录下回看到一个 Modelsim.ini 的文件。打开文件，拷贝文件(第 47 行一直到“[vcom]的上一行”)。接着在 Modelsim 的安装目录里面，找到 Modelsim.ini 后打开(先修改属性，去掉只读)。在第 12 行的行尾，回车换行，将前面复制的直接拷贝上去。其他内容不要更改。保存。

库生成了，路径也安排好了，下面就是直接级联了。打开 ISE14.7，Edit→Preferences, 打开 Preferences 窗口。选择需要仿真的路径，设置完成后，直接点击 OK。



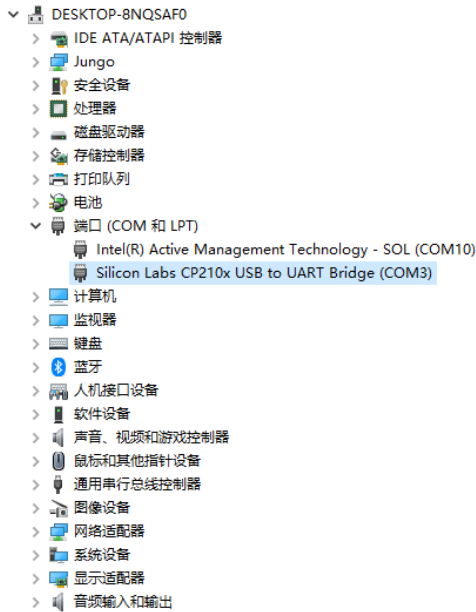
上面几个部分主要讲解的是 ISE14.7、ModelSimSE 的安装、破解以及两个软件的级联配置。如果需要将软件编译的 BIT 文件下载到 FPGA 中，还需要一个步骤就是连接下载器，进行下载。

1.4 CP2102 串口芯片驱动安装

在 CP2102 官网最新的驱动，或网盘下载 UART 驱动，双击 CP210xVCPInstaller_x64.exe 进行安装，单击“下一步”按钮直到安装完成。

名称	修改日期	类型	大小
x64	2016/5/4 20:42	文件夹	
x86	2016/5/4 20:42	文件夹	
CP210x_Windows_Drivers.zip	2017/1/14 22:28	WinRAR ZIP 压缩...	3,767 KB
CP210xVCPInstaller_x64.exe	2016/3/28 22:38	应用程序	1,034 KB
CP210xVCPInstaller_x86.exe	2016/3/28 22:38	应用程序	911 KB
dpinst.xml	2016/3/28 22:32	XML 文档	12 KB
SLAB_License_Agreement_VCP_Windo...	2016/3/28 22:32	文本文档	9 KB
slabvcp.cat	2016/5/2 23:59	安全目录	11 KB
slabvcp.inf	2016/5/2 23:53	安装信息	12 KB

将 TEC-PLUS 核心板通过 USB 下载线连接到 PC，给 TEC-PLUS 核心板供电。按下 SW1 开关，LED7 灯亮，说明开发板供电正常。以 Windows 10 系统为例，在桌面的“此电脑”图标上右击，选择“管理”，计算机管理-设备管理器-端口(COM 和 LPT)。如果没有安装驱动，参考上面的程序安装驱动即可。

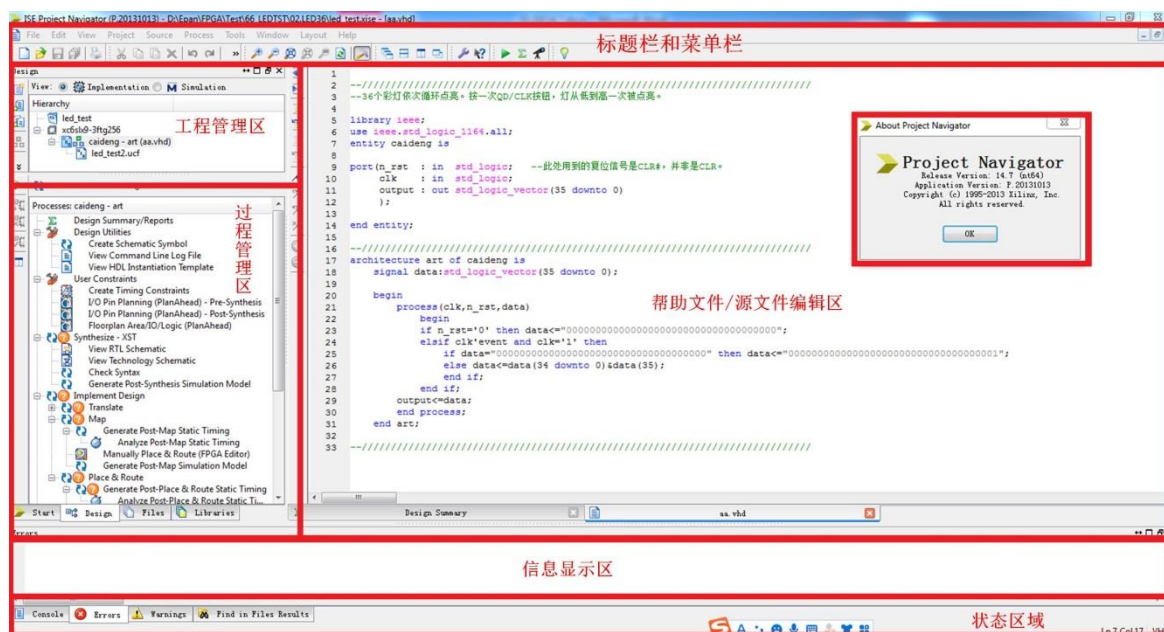


需要说明的是：不同 PC 上的 COM 端口可能不一样，但是一般都会有 Silicon Labs CP210X usb UART，说明核心板上面的串口已经被识别。

1.5 ISE14.7 简介与教程

本章只简单介绍 XILINX FPGA 开发所需要的简单流程。ISE 14.7 的主要功能包括设计输入、综合、仿真和下载，含 FPGA 开发的全部过程，从功能上讲，可以不借助第三方的 EDA 软件，也可以和其他的仿真软件相互关联。

打开 ISE 14.7 软件，如图所示：ISE 14.7 的主窗口

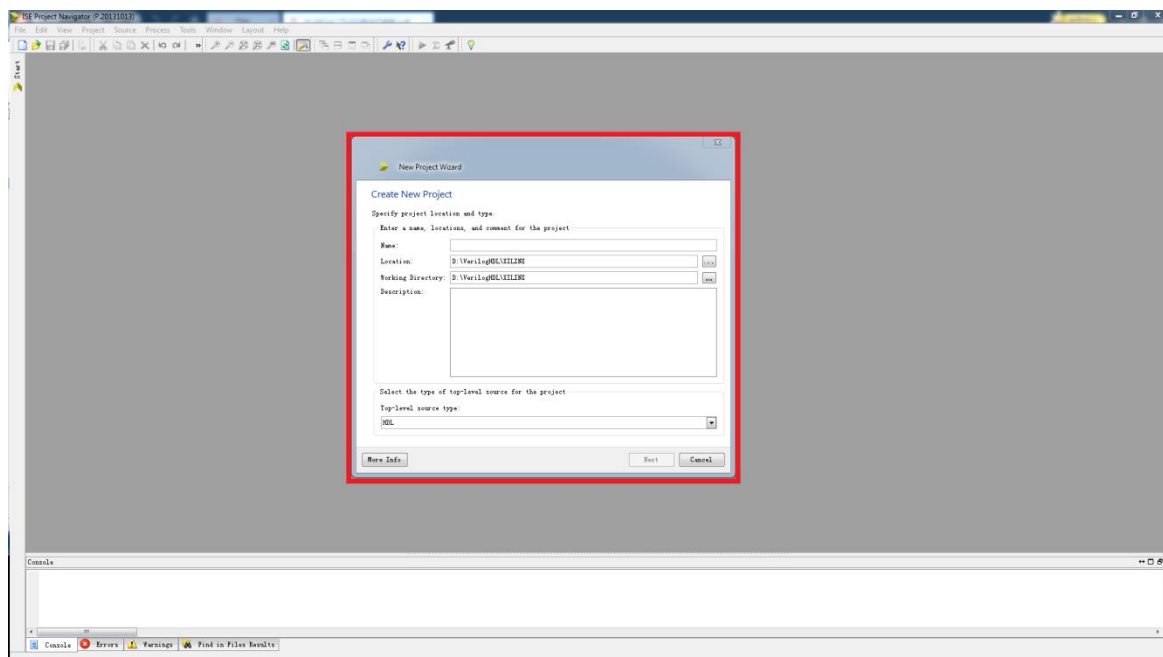


首先打开 ISE14.7 软件，每次启动时，ISE 14.7 都会默认到最近使用过的工程界面。

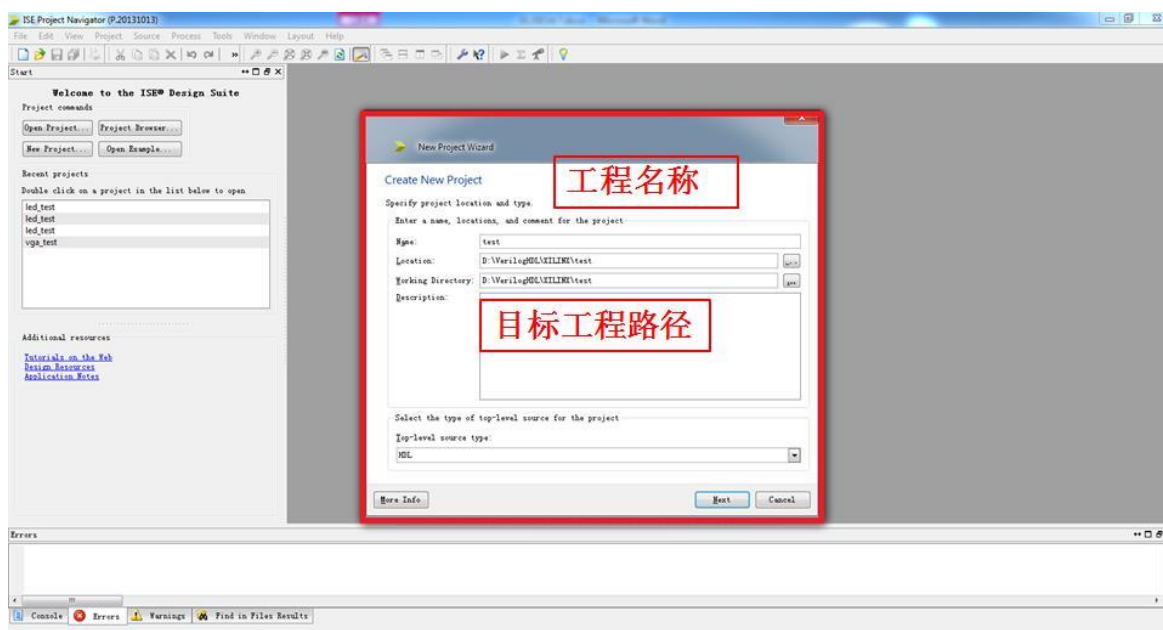
下面以一个彩灯循环点亮程序的设计为例，说明如何使用 ISE 14.7 创建工程并进行调试以及如果使用 XC6SLX9-3FTG256 进行配置。

1.1 新建工程

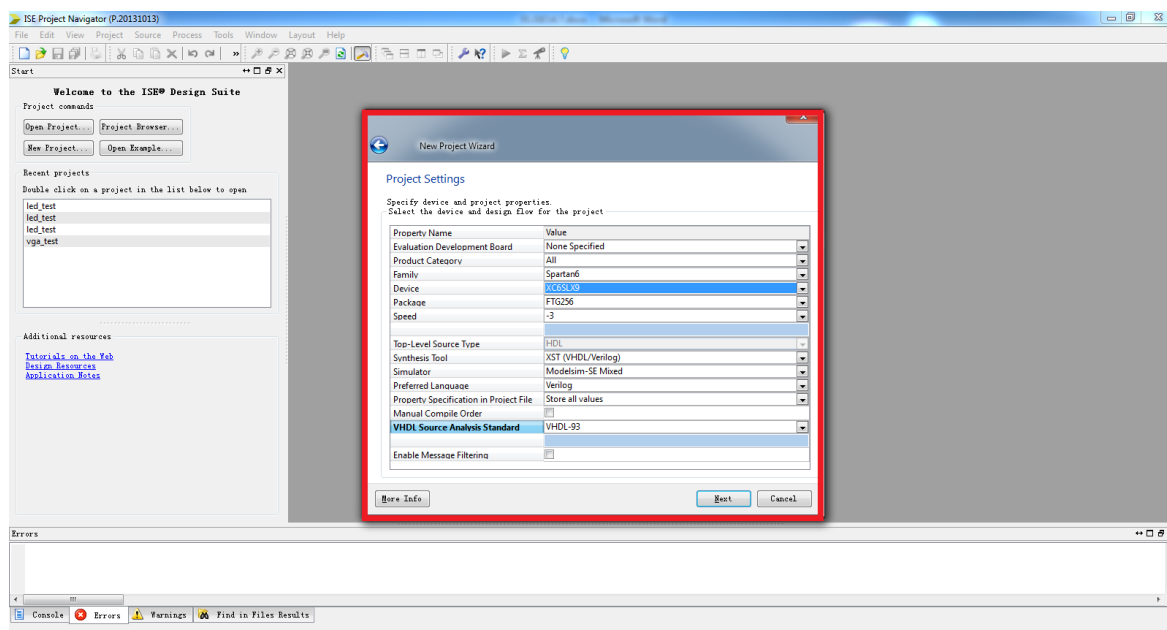
步骤一：在 ISE Project Navigator 开发环境里选择菜单 File->New Project。



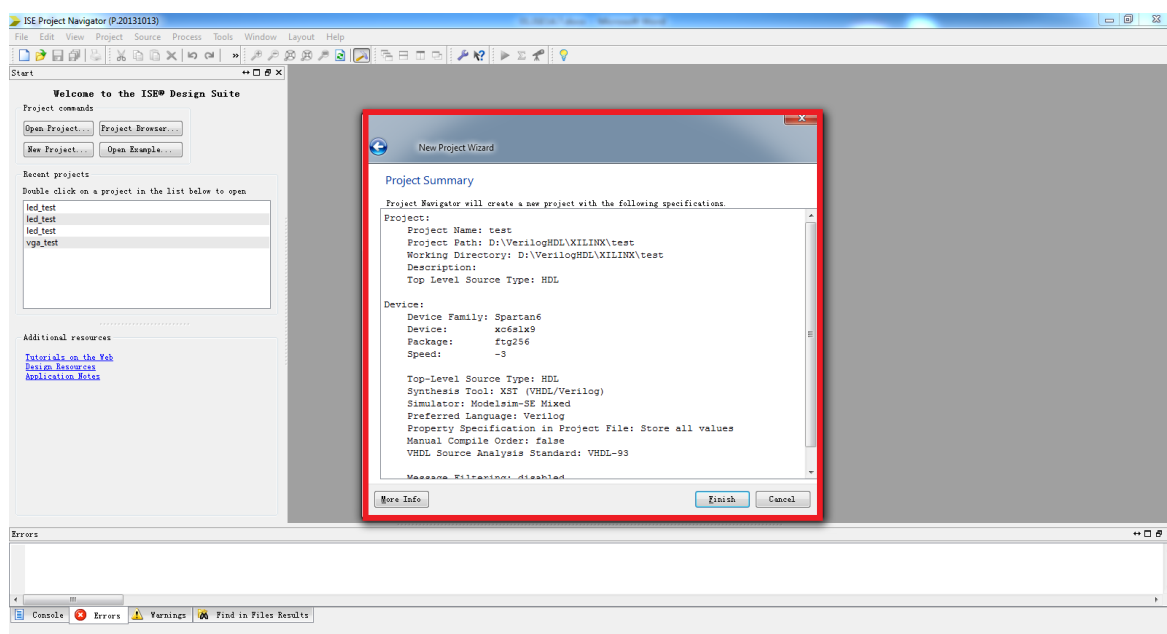
步骤二：在弹出的对话框中输入工程名称和目标工程路径，我们这里取一个 test 的工程名，目标工程路径可以自己选择，选择 Next 按钮。（注意：在所有工程文件的建立和保存过程中，保存路径和文件名称等都不能使用中文、空格或者其他特殊字符，以免造成编译不过或报错等现象）；



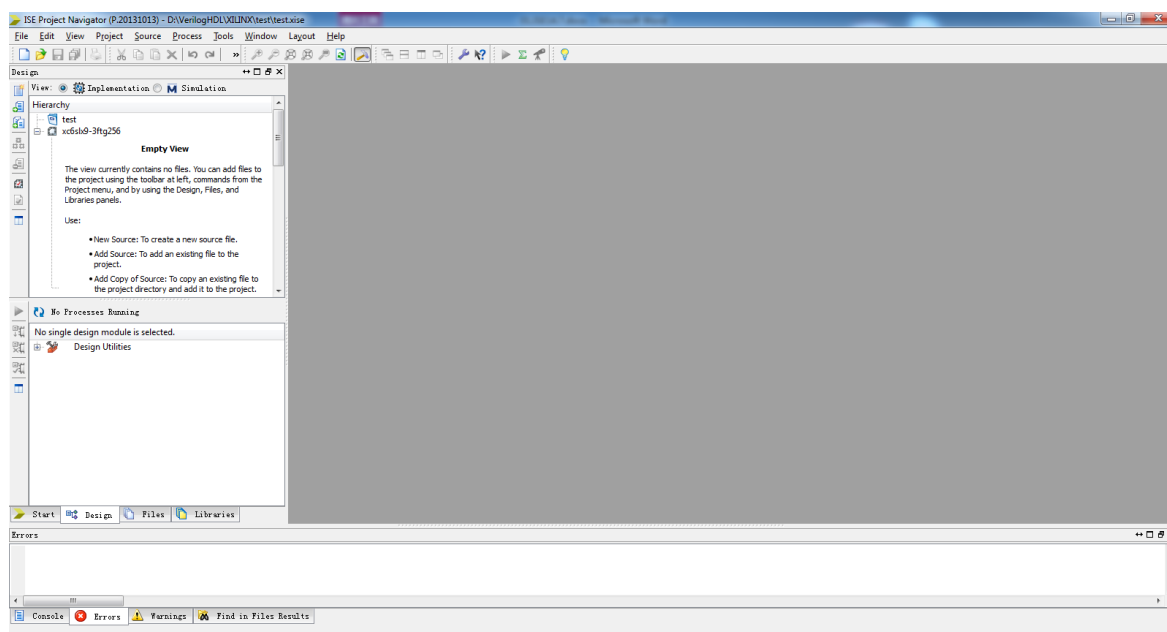
步骤三：在接下来的对话框中选择试验箱核心板所用的 FPGA 器件型号已经进行工程的一些配置。这里可以指定所选用的芯片，主要包括芯片系列、型号、封装以及速度等，这些内容在所选的芯片上都有标注，根据芯片选择即可，这里如下图所示的 Spartan6 系列的 XC6SLX9, 封装为 FTG256, 速度等级为-3, Preferred Language 对应语言类型，使用 Verilog/VHDL 语言编程均可以。



步骤四：单击 Next 按钮，出现工程汇总对话框，里面显示了所创建工程的一些信息，如下图所示。



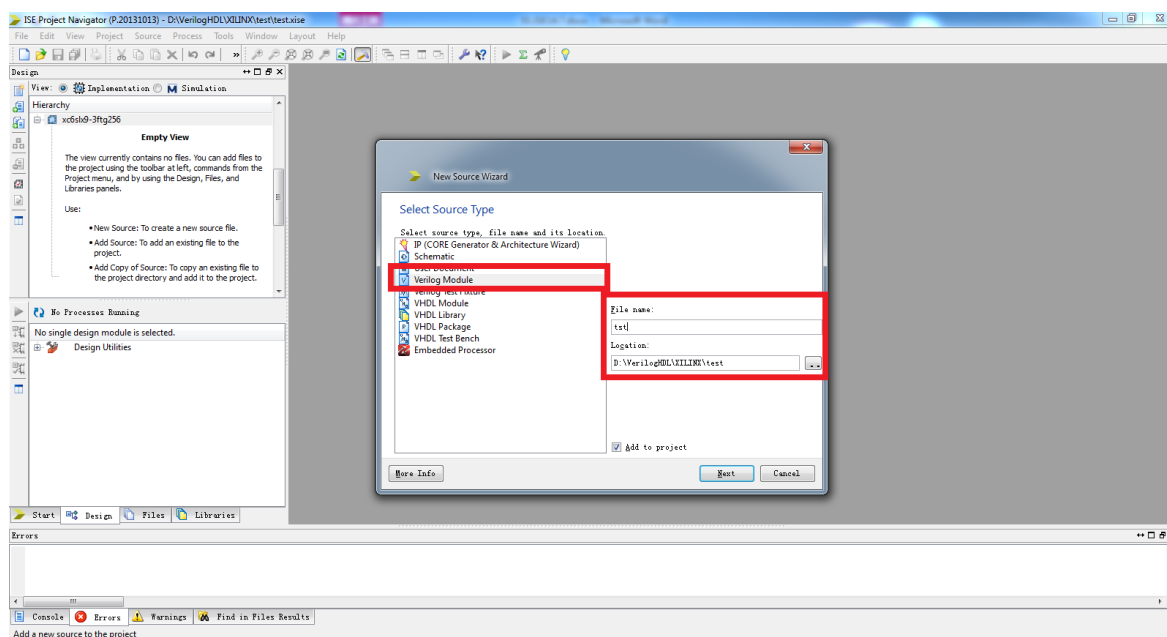
步骤五：单击 Finish 按钮，完成创建空白工程，在管理区可以看到新创建工程，如图所示。



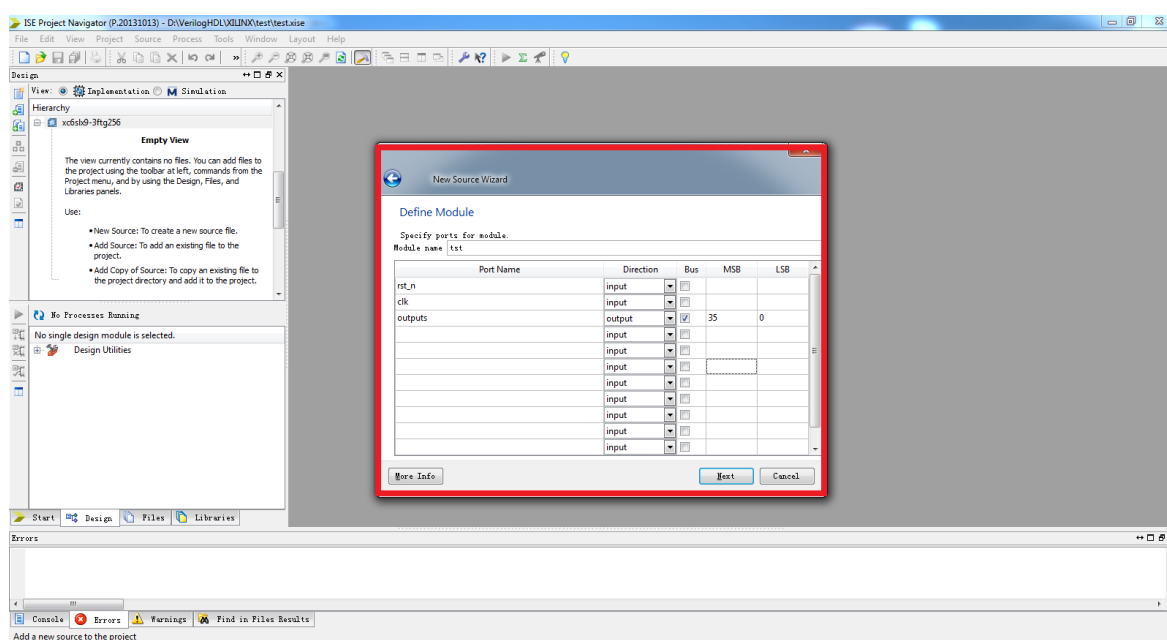
简单的工程建立完毕，现在需要给新建的工程中新建源代码。

1.1.1 新建源代码

下面开始给新工程创建并添加源文件，在管理区内右击工程 Ledtst，选择“New Source”，出现创建源文件向导的选择源文件类型对话框，如图所示，在此选择“Verilog Module”，在 File Name 文本框中输入“test”，下面有个复选框选择将文件添加到工程中。

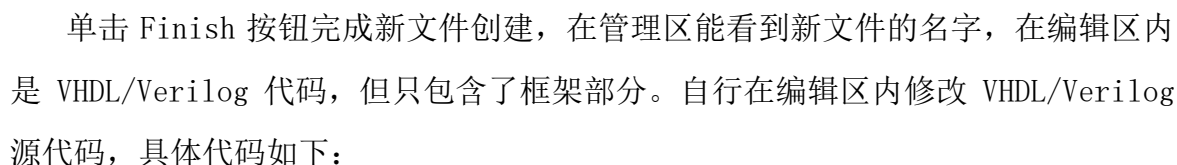


单击 Next 按钮，出现模块定义对话框，如图所示，这里实现一个这是一个 36 位彩灯依次循环点亮程序，单步模式下每次点亮一个 LED 灯，36 位 LED 灯全部点亮后从第一位循环开始。

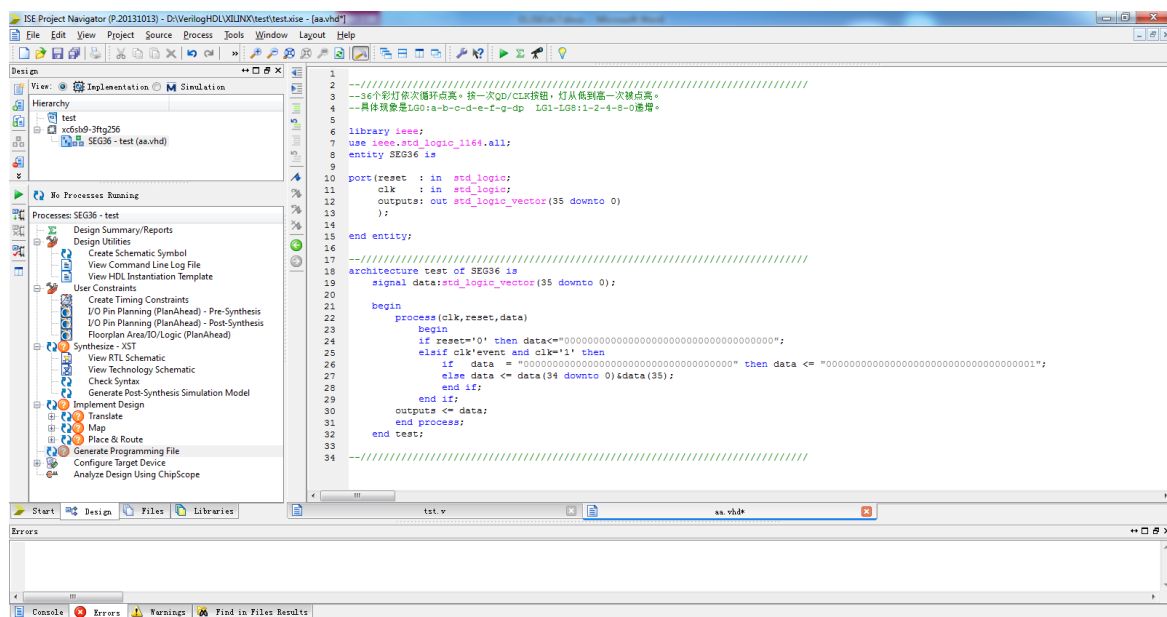


其中，“Module Name”就是输入的“test”，下面的列表框用于对端口的定义。“Port Name”表示端口名称，“Direction”表示端口方向(可以选择 input、output 或 inout)，“MSB”表示信号的最高位，“LSB”表示信号的最低位。单位信号的 MSB 和 LSB 不用填写。在端口定义对话框中可以先不做任何定义，直接 Next。

单击 Next 按钮，出现新文件总结对话框，如图所示。



编写代码后保存，自动生成为工程的 Top 文件，新文件添加完成如下图所示：



1.1.2 添加源代码

小工程，所以代码直接在上一步添加完成。

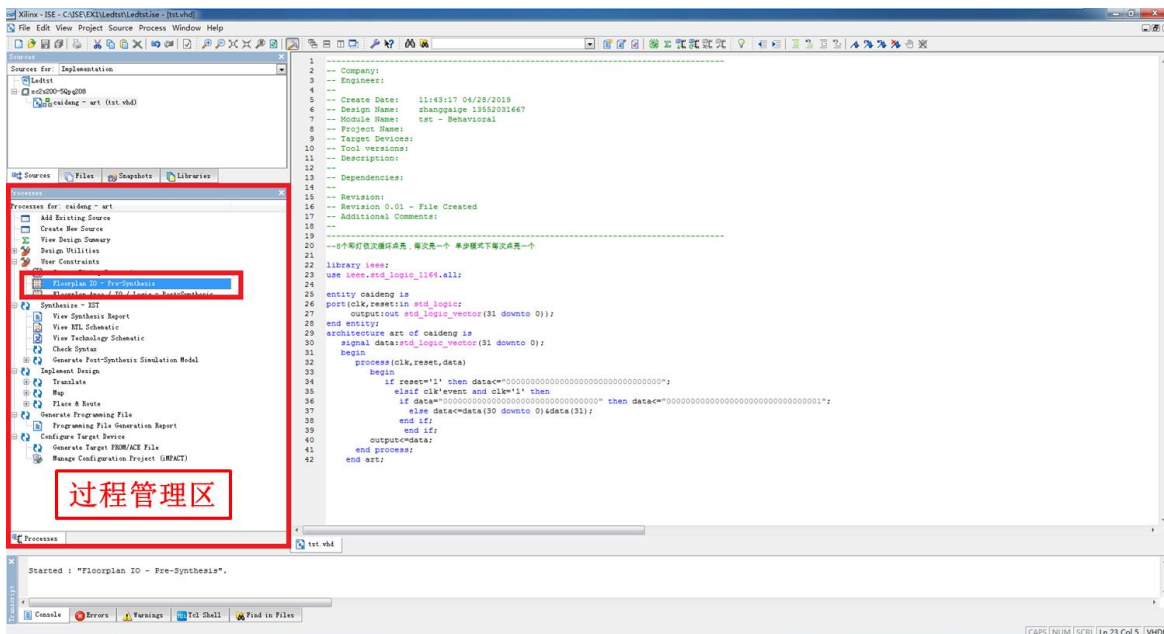
1.1.3 代码模板

ISE 中内嵌的语言模块包含了大量的开发实例和所有 FPGA 语法的介绍和举例，包含 Verilog HDL/HDL 的常用模块、FPGA 原语使用实例、约束文件的语法规则以及各类指令和符号的说明。语言模块不仅可在设计中直接使用，还是 PFPGA 开发的最好的工具手册。

1.2 添加约束

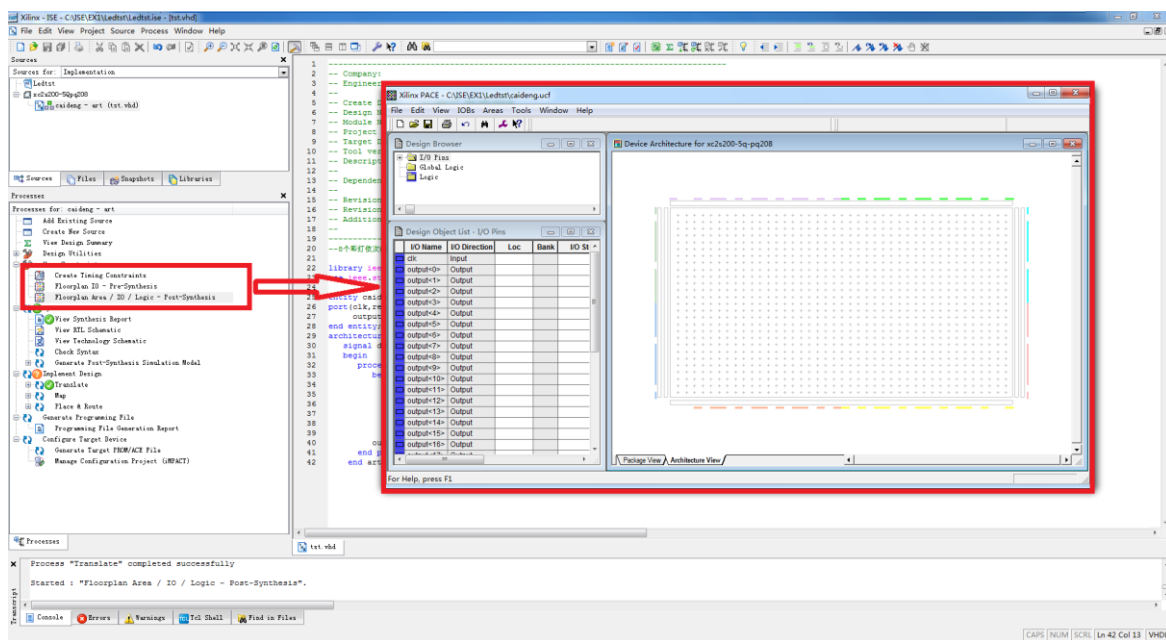
约束是 FPGA 设计所不可缺少的。例如，管脚约束将模块的端口和 FPGA 的管脚对应起来。在 ISE 中有很多种约束，它们可以指定设计各个方面的设计要求。通过附加约束可以控制逻辑的综合、映射、布局和布线，以减少逻辑和布线延时，从而提高工作频率，还可以指定 I/O 引脚所支持的接口标准和其电气特性等，本节只介绍如果对设计进行管脚约束。这里我们需要对 Ledtst.vhd 程序中的输入输出端口分配到 FPGA 的真实管脚上，这里需要准备一个 FPGA 的引脚绑定文件.ucf 并添加到工程中。

在管理区的“View”栏中选择“Implementation”项，选择要约束的源文件，这里是“SEG36.vhd”，在下面的操作窗口中展开“User Constraints”，会出现用户约束设定的选项，如图所示。



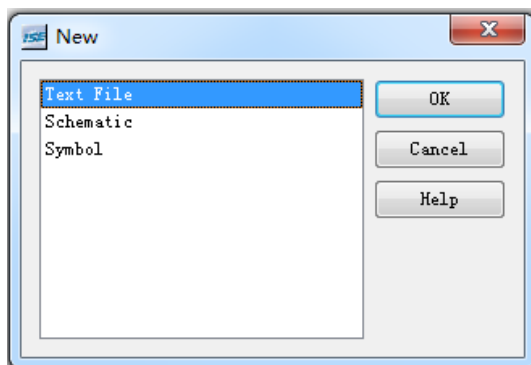
双击“I/O Pin Planing-Post-Synthesis”，会出现添加用户约束文件的对话框，单击 YES 按钮，确认添加用户约束文件，打开管脚约束窗口。

在其中可以指定设计的管脚，在左侧的窗口中选择 I/O Port 标签栏，会出现设计中所有的 I/O 管脚，展开后在 Site 列中就可以指定对应的管脚号，直接单击输入即可。管脚指定完成后保存退出窗口，在管理区的 tst.vhd 文件下会出现约束文件 tst.ucf。



用户也可以直接编辑该文件进行约束设定：

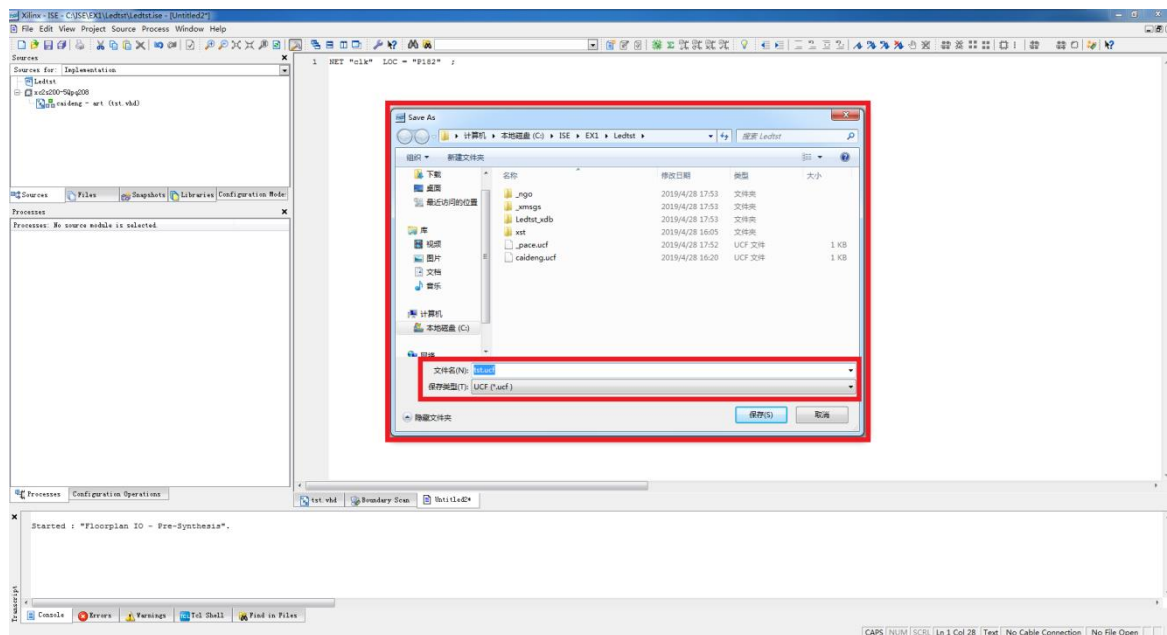
步骤一 新建一个空白文件 File->New,选择 Text File。



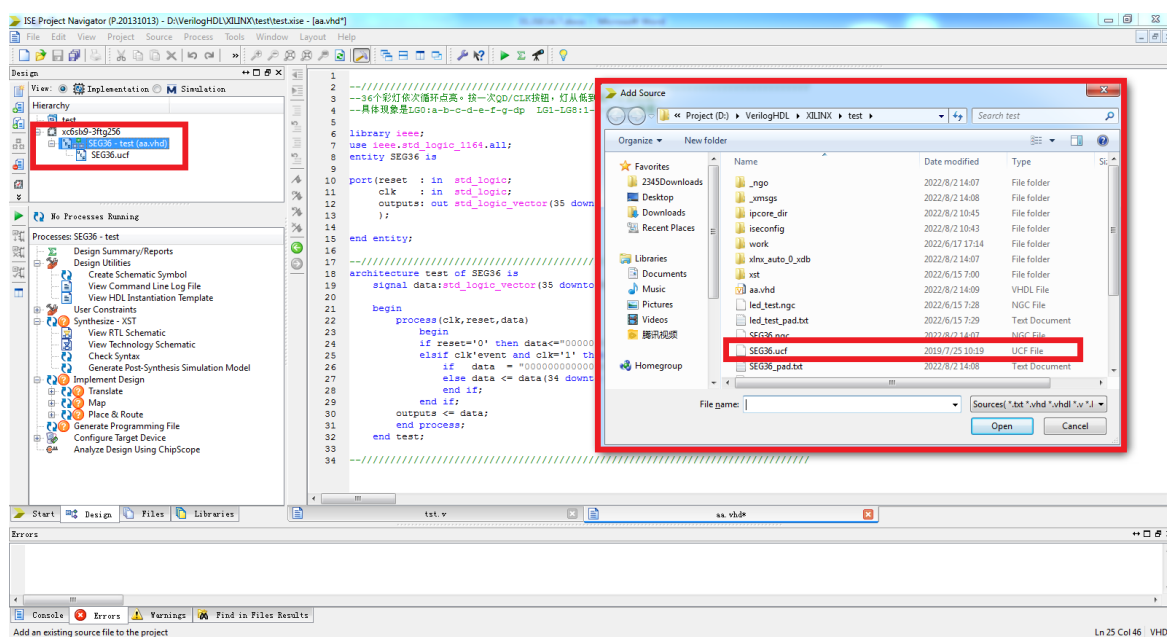
步骤二 在这里 Text 文件中添加以下的引脚定义

基本的 UCF 编写的语法，普通 I/O 口只需要约束引脚号和电压；NET “端口名称” LOC = 引脚编号 | IOSTANDARD = “电压”；例如：NET “clk” LOC = “J16”；需要注意的是，ucf 文件是大小敏感的，端口名称必须和源代码中的名字一致，且端口名字不能和关键字一样。但是关键字 NET 是不区分大小写的。

步骤三 完成后另存为 SEG36. ucf 文件



步骤四 把 SEG36. ucf 文件添加到工程中，点击 Project->Add Source, 添加后如下图所示：



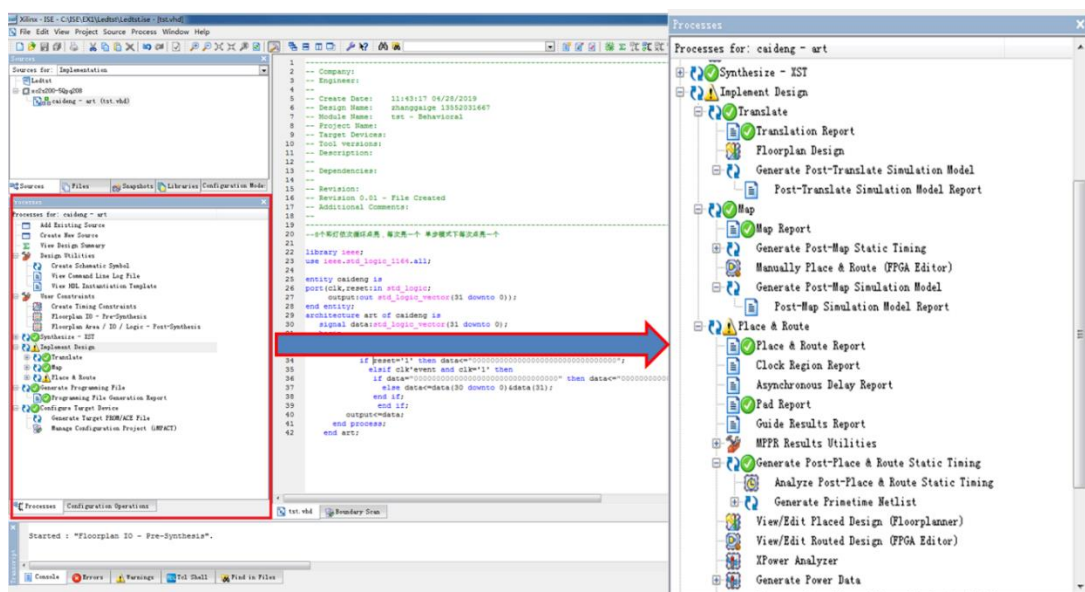
1.3 综合与实现

保存项目并且开始编译, 综合与实现就是将逻辑网表翻译成底层模块与硬件原语, 将设计映射到器件结构上, 进行布局布线, 分成翻译(Translate)、映射(Map)、布局布线(Place&Route)三步:

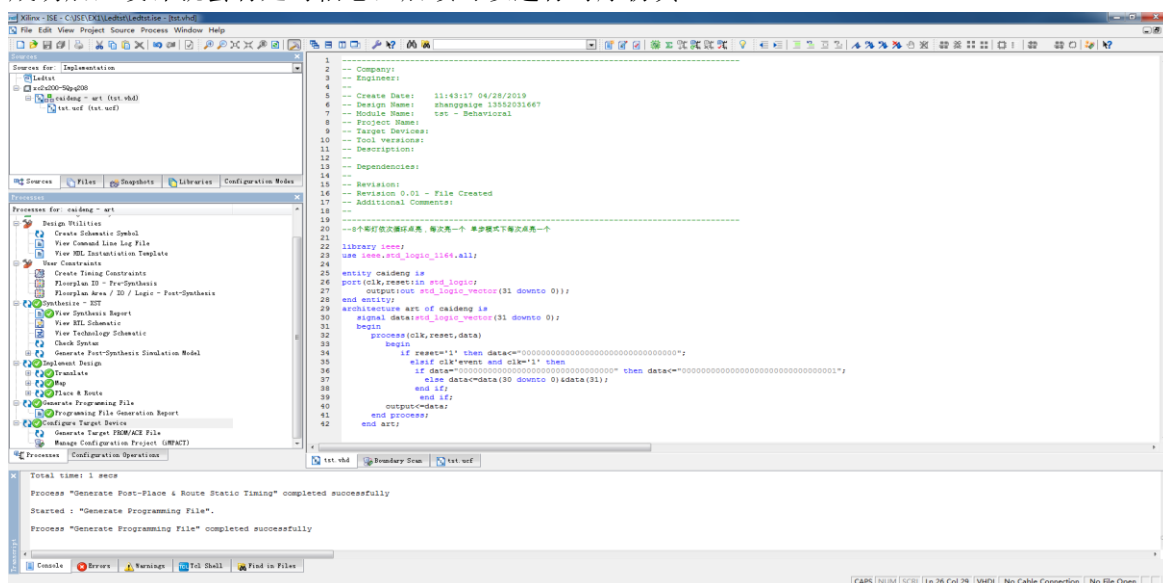
(1) 翻译: 把多个设计文件合并成一个单独的网表设计。

(2) 映射: 把网表中的门级逻辑映射到物理器件资源上。

(3) 布局布线: 把 Map 中的物理器件资源在器件上布局, 并用布线资源连接起来, 把时序数据写入到时序报告中。

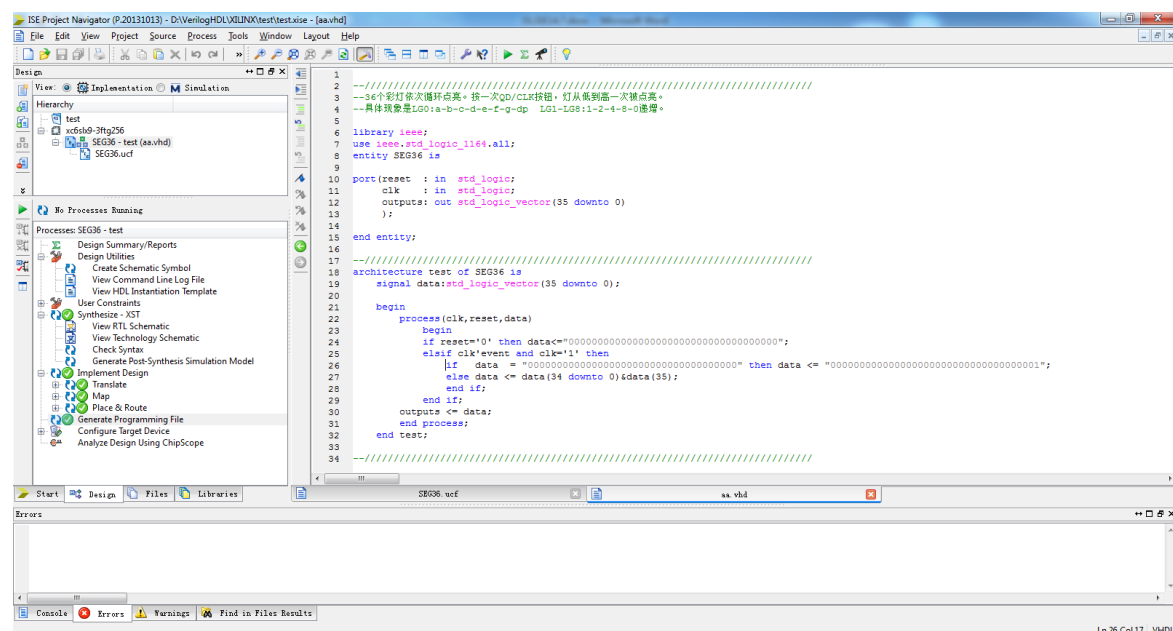


在管理区中选择 SEG36.vhd, 在操作栏中选择 implement Design, 双击开始实现, 实现成功后, 设计就会有延时信息, 后续可以进行时序仿真。



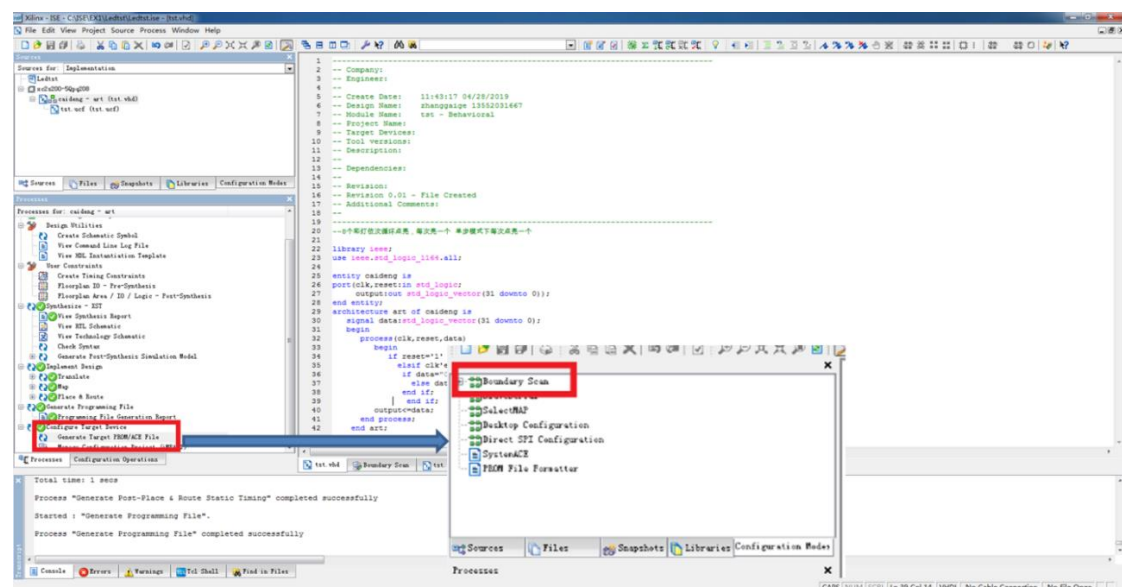
程序编写和管脚分配完成之后, 需要综合, 在软件左侧 Process 栏, 选择 Process, 双

击 Synthesize Design 对设计进行编译工程，编译完成完成后 Synthesize Design 显示绿色对勾(如果显示红色叹号，说明代码有问题，根据提示修改代码)。编译成功后在 Console 窗口出现成功的消息。



1.4 程序下载

经过前面的编译和综合，我们可以把 Bit 文件下载到 FPGA 芯片中，看一下程序实际运行的效果，随后开始实际测试了，首先在配置之前需要生成配置文件，并连接到下载电路。双击操作栏中的“Generate Programming File”，生成可配置文件，将 USB 下载线分别和计算机接口及实验仪的 JTAG 口相连接，首次连接计算机会自动安装驱动程序，然后打开设备供电。双击操作栏的 Configure Target Device，会弹出询问是否打开 IMPACT 的对话框，单击 OK 按钮，将打开 IMPACT 窗口。



第二部分：板载资源简介与使用

2.1 按键监测或译码电路

➤ 目的

- 熟悉和掌握 FPGA 开发流程;
- ISE14.7 软件和 Mmodelsim SE 使用简单方法;
- 通过实验理解译码器电路;
- 学习 VHDL 或 Verilog HDL 行为级描述方法描述组合逻辑电路。

➤ 任务

设计一个 2-4 译码器电路。

➤ 原理

2-4 译码器，输入的 2 位二进制代码共有 4 种状态，译码器将每个输入代码译成对应的一根输出线上的高低电平信号，输出低电平有效。将输入的 A、B 和输出的 Y0、Y1、Y2、Y3 的关系写成逻辑表达式则得到：

B	A	Y3	Y2	Y1	Y0	备注
X	X	1	1	1	1	
0	0	1	1	1	0	
0	1	1	1	0	1	
1	0	1	0	1	1	
1	1	0	1	1	1	

➤ VHDL 或 Verilog HDL 建模描述

■ VHDL 程序源代码

```
1  -----
2  library IEEE;
3  use IEEE.STD_LOGIC_1164.ALL;
4
5  entity Decoder139 is
6      Port ( A : IN  STD_LOGIC_VECTOR (1 downto 0);
7            Y : OUT STD_LOGIC_VECTOR (3 downto 0));
8  end Decoder139;
9
10 architecture Behavioral of Decoder139 is
11 begin
12     process(A)
13     begin
14         case(A) is
15             when "00" => Y <= "1110";
16             when "01" => Y <= "1101";
17             when "10" => Y <= "1011";
18             when "11" => Y <= "0111";
19             when OTHERS=> Y <= "XXXX";
20         end case;
21     end process;
22 end Behavioral;
23 -----
```

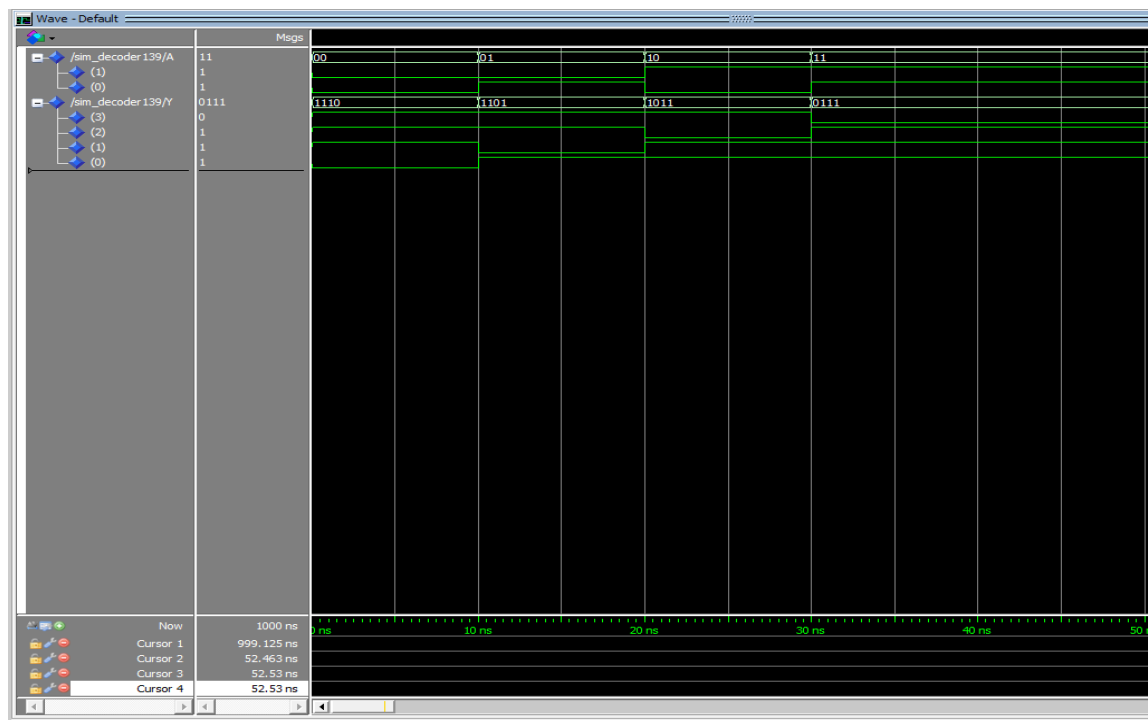
■ VHDL 仿真源程序


```

1  -----
2  LIBRARY ieee;
3  USE ieee.std_logic_1164.ALL;
4
5  ENTITY Sim_decoder139 IS
6  END Sim_decoder139;
7
8  ARCHITECTURE behavior OF Sim_decoder139 IS
9      COMPONENT Decoder139
10         PORT(A : IN std_logic_vector(1 downto 0);
11              Y : OUT std_logic_vector(3 downto 0));
12     END COMPONENT;
13     signal A : std_logic_vector(1 downto 0);
14     signal Y : std_logic_vector(3 downto 0);
15 BEGIN
16     uut: Decoder139 PORT MAP (
17         A => A,
18         Y => Y);
19     Y_proc: process
20     begin
21         A <= "00"; wait for 10 ns;
22         A <= "01"; wait for 10 ns;
23         A <= "10"; wait for 10 ns;
24         A <= "11"; wait for 10 ns;
25         wait;
26     end process;
27 END;
28 -----

```

➤ 仿真图形



➤ 实验步骤

- 打开 ISE14.7 软件，建立工程。
- 新建 VHDL 或 Verilog HDL 设计文件，并设计功能代码和仿真代码。
- 综合并且分配管脚，将输入信号分配到数据开关，将输出信号分配到 LED 灯。
- 构建并输出编程文件，下载编辑文件到 FPGA 中。
- 按下对应按键/数据开关，观察输出结果。

EX:可以直接下载 74LS139 译码器工程文件，需要单独接排线电平开关到 J10A（最右侧 2 位可用 I/O）。电平开关 K 最低两位做为输入，核心板 LED3-LED0 做为输出显示。

2.2 流水灯

需求：复位情况下，4 个 LED 灯全灭。当复位按键放开后，首先点亮第一个灯；一秒钟后第一个灯熄灭，同时点亮第二个灯；一秒钟后第二个灯熄灭，同时点亮第三个灯，如此功能直到第四个灯熄灭，同时点亮第一个灯。如此循环，实现流水。

➤ VHDL 或 Verilog HDL 建模描述

■ 程序源代码

```
1 //=====
2 `timescale 1ns / 1ps
3
4 module led_test ( clk,          // 50Mhz
5                  rst_n,        // 复位按键
6                  led );        // LED4--LED0
7 //=====
8 input clk;
9 input rst_n;
10 output [3:0] led;
11 reg [31:0] timer;
12 reg [3:0] led;
13 //=====
14 always @(posedge clk or negedge rst_n) //
15 begin
16     if (~rst_n) //
17         timer <= 0; //
18     else if (timer == 32'd199_999_999) //50MHz4 (50M*4-1=199_999_999)
19         timer <= 0; //
20     else
21         timer <= timer + 1'b1; //
22 end
23 //=====
24 always @(posedge clk or negedge rst_n) //
25 begin
26     if (~rst_n) //
27         led <= 4'b0000; //LEDLED
28     else if (timer == 32'd49_999_999) //
29         led <= 4'b0001; //LED1
30     else if (timer == 32'd99_999_999) //
31         led <= 4'b0010; //LED2
32     else if (timer == 32'd149_999_999) //
33         led <= 4'b0100; //LED3
34     else if (timer == 32'd199_999_999) //
35         led <= 4'b1000; //LED4
36 end
37 endmodule
38 //=====
```

程序仿真非常简单，此处一笔带过。

实现现象是：按下核心板上面的复位按钮，LED4-LED1 全部熄灭，松开复位按键，先点亮 LED1，一秒钟后 LED1 熄灭同时 LED2 点亮，依次循环，周而复始。

2.3 串口 Uart 通信

Uart 是一种通用的串口数据总线,用于异步通信,也可以和 MAX232 配合进行 USB 转 COM 的通信。在 FPGA 开发设计中, UART 用来和 PC 进行通信, 本实验主要监测了解通信协议和传输时序。

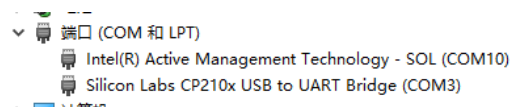
TEC-PLUSV3.0 核心板上电源口 J5 既可以作为单独使用开发板情况下提供电源, 又可以作为开发板和 PC 之间的串口通信。硬件设计采用 CP2102 芯片作为 USB 和 UART 电平转换的桥梁, USB 接口采用 MINI_USB 接口, 学生可以使用设备自带的 USB 连接开发板到 PC 上进行串口的数据通信。

CP2102 和 FPGA 连接关系:

信号	PIN	具 体 意 义	备注
RXD	D5	FPGA 数据 UART 接收, 而对 CP2102 芯片来说是数据发送	
TXD	D6	FPGA 数据 UART 发送, 而对 CP2102 芯片来说是数据接收	

USB 转 UART 驱动安装

此开发板所用 USB 转 UART 芯片是 CP2102, 驱动程序在驱动光盘里, 可以直接安装。安装完成后入下图所示: 本机选择的是 COM3 口, 用户可以按照自己的 PC 端口自我选择。



本历程的主要功能是演示核心板的 UART 数据接收和发送的功能, 在程序没有接收到 PC 机发来信息的时候, 串口会不断的通过串口向 PC 发送: HELLO TEC-PLUSV3.0 这一组字符串。当用户从 PC 机发送数据给核心板, 程序接收到数据会把数据从串口发回给 PC, 从而实现 LOOPBACK 的功能。

编译生成 .bit 文件, 再下载到 FPGA。用 USB 线连接 PC 和开发板的 J5, 打开串口调试助手, 选择上面列表中对应的 COM 口, 波特率 9600, 校验位设置为奇校验或偶校验, 停止位为 1。打开电脑后可以看到显示窗口从 FPGA 不断发来的 HELLO TEC-PLUSV3.0 这一组信息。如果从串口调试助手发送数据, 可以看到 FPGA 会立马回传同样的信息给 PC。

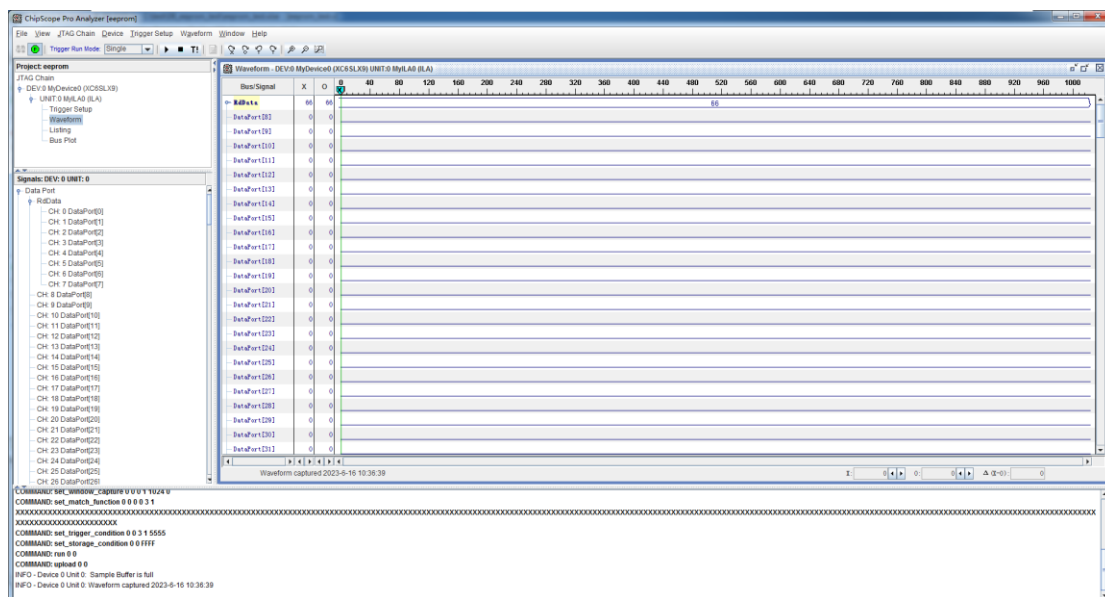
2.4 EPPROM 读写

开发板上有一个 IC 接口的 EEPROM 芯片 24LC04，容量大小为 4Kbit，用户可以用来存放一些硬件设置数据或者用户信息。EEPROM 的主要特性就是掉电不丢失存储芯片的数据，用户可以通过总线对他进行多次编程。

功能实现：上电后写一个数据到 EEPROM 的地址 0，再读出地址 0 的内容。此处写入的数据是 0X66。

写完硬件描述语言，生成.bit 文件，下载到 FPGA 中，然后打开 chipscope 来观察下从 EPROM 地址 0 读出的数据是否为 0X66。

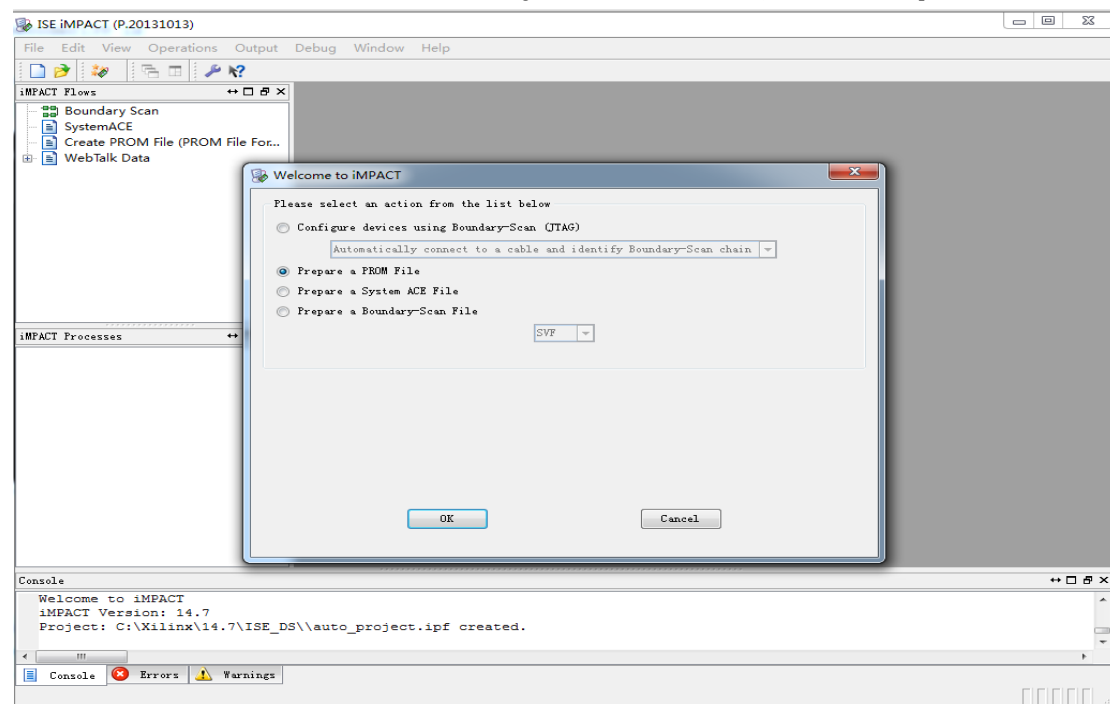
从程序开始 XILINX 工具目录下打卡 Chipscope, 点击 Open cable 按钮建立 JTAG 连接。开发板和 JTAG 连接正常的话, chipscope 可以找到开发板所使用的 FPGA 芯片。点击 OK 即可。在 DATA Port 里的 CH0-CH7 组合成一组命名为 RdData。点击 Apply Setting and Arm Trigger 按钮查看 RdData 的数据。可以看到从 EEPROM 地址 0 读出的 RdData 的数据为 0X66。



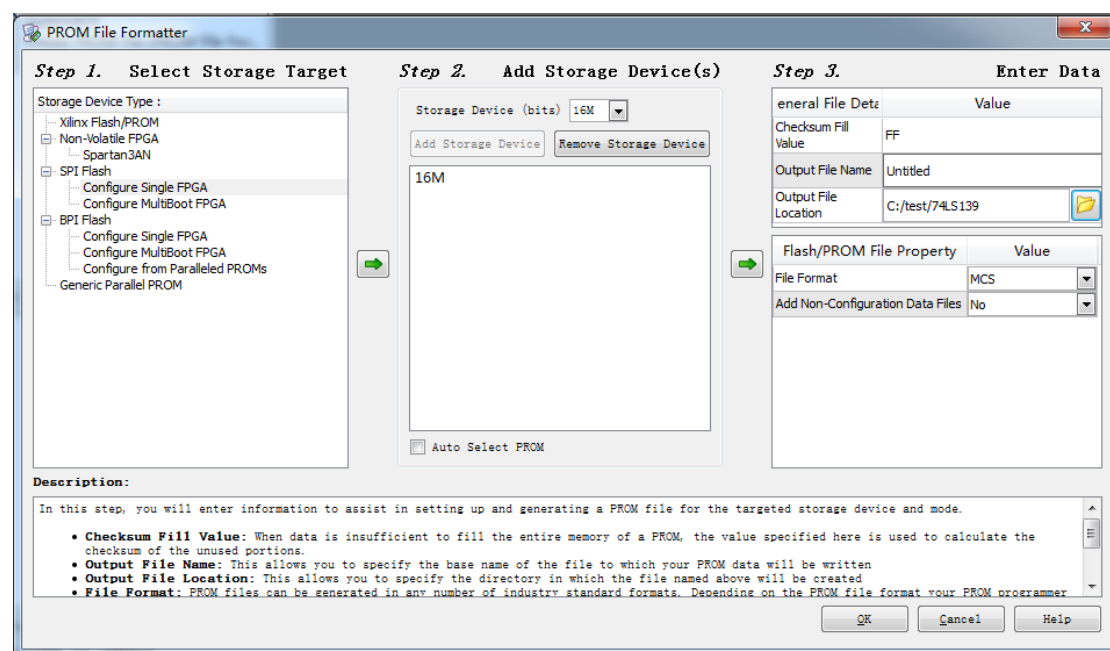
2.5 FLASH 读写

本开发板配置了一块 16Mbit 的 FLASH, 可以用来存储配置程序。FPGA 可以直接通过 USB 下载器进行下载, 但是 FLASH 的配置过程和 FPGA 是不同的, 所以这一节咱们简单介绍下, 方便后续使用。

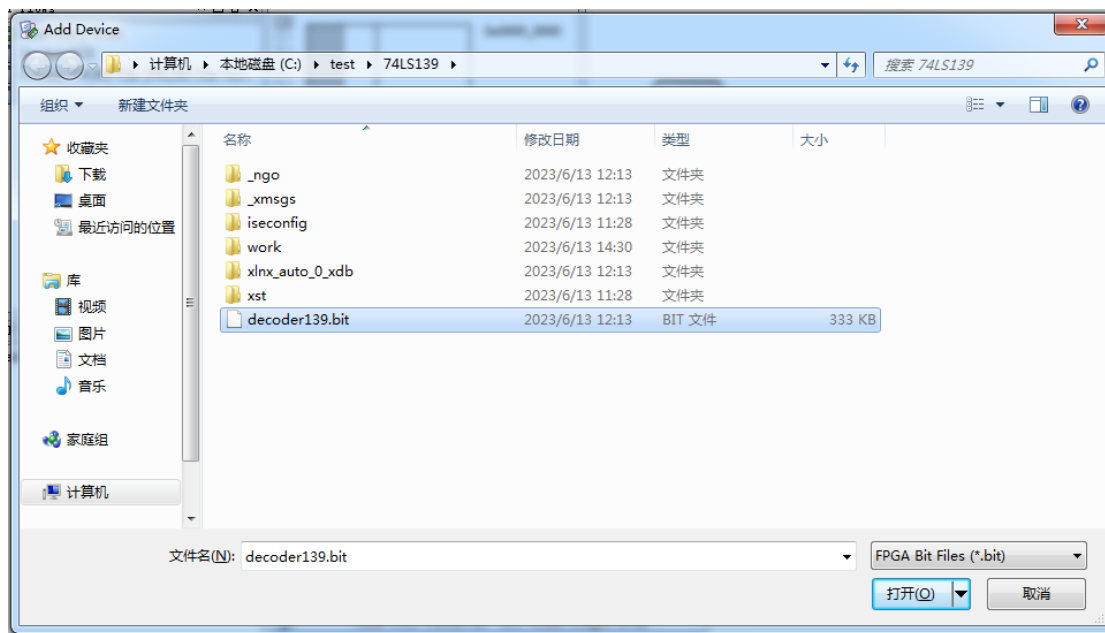
在 ISE IMPACT 中选择 File-->New Project, 在弹出的对话框中选择 Prepare a PROM File



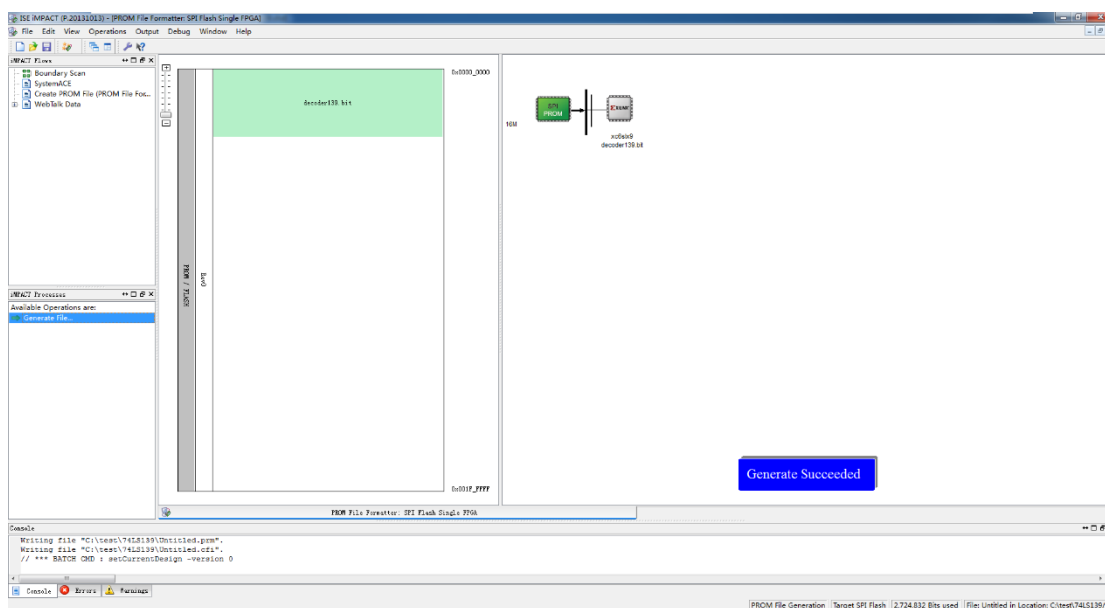
生成 mcs 文件之前需要确定 FLASH 的型号以及容量和 MCS 文件的存放目录、名称。在弹出的 PROM File Formatter 界面中的依次选择如下:



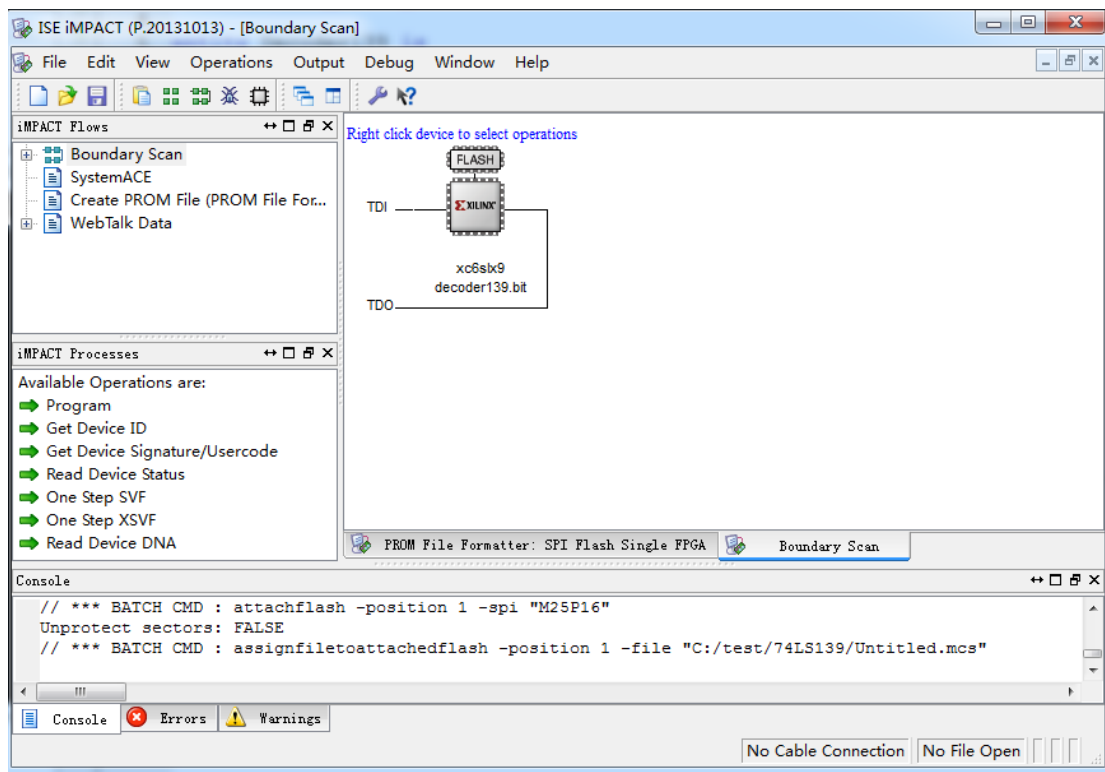
点击 OK 弹出对话框, 直接点击 OK, 之后弹出窗口选择已经生成的 .bit 文件, 直接加载。



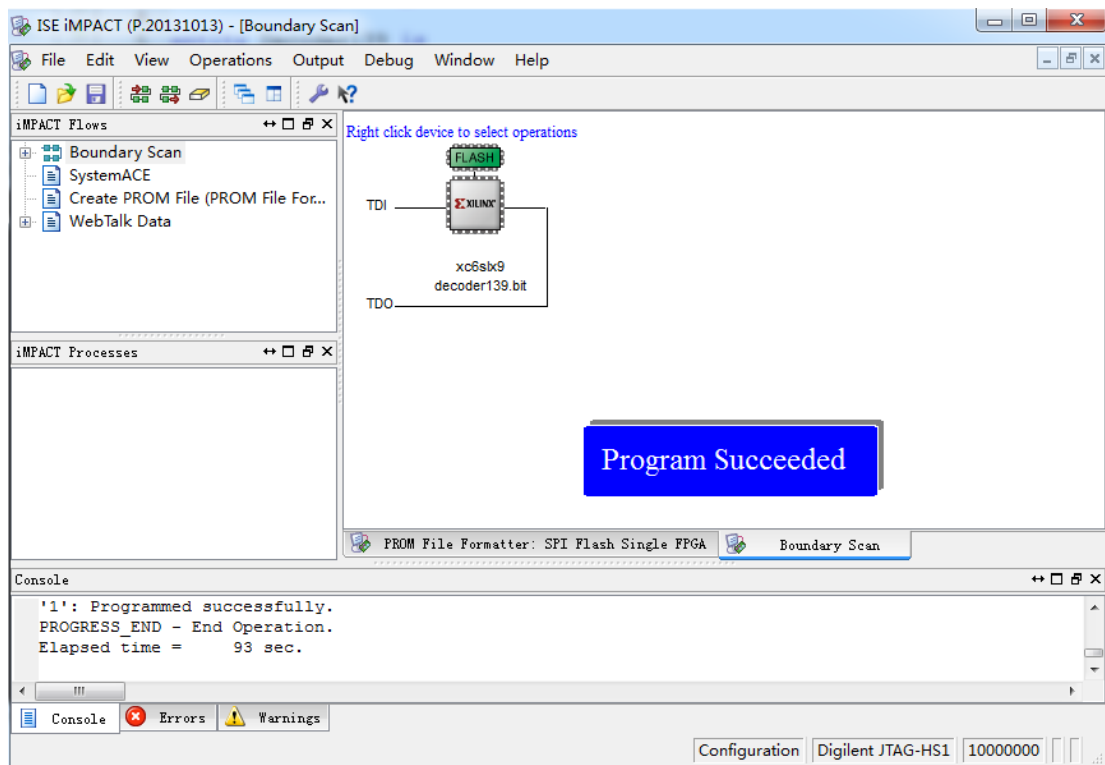
加载完成后，双击左边界面中的生成文件，就可以生成 MCS 文件。



MCS 文件生成成功后在界面会显示”Generate Succeeded”。截止到这一步，MCS 生成成功。和下载 FPGA 的方法类似，打开 IMPACT FLOWS 界面下的 Boundary Scan 这个界面，加载完成需要下载的.bit 文件，在器件 的上方有虚线，双击选择合适的型号：M25P16。



选中 FLASH 图标，右击选择 Program 进行编程。烧写的时间和配置的文件的大小有直接的关系，所以稍等片刻。烧写成功，会弹出成功的界面。



截止到这一步，FLASH 写入完毕，程序已经固化到 FLASH 中，关机-开机测试下，看核心板是否可以继续运行刚才的测试逻辑。

2.6 SDRAM 读写

SDRAM 同步动态随机存储器，因为 SDRAM 具有存取速度大大高于 FLASH 存储器，并且具有读写的属性，因此 SDRAM 在系统中主要用于程序的运行空间，大数据的存储及堆栈。

本开发板采用的 SDRAM 是 HYNIX 公司的 HY57V2562 型号，容量为 256Mbit，采用 54 引脚的 TSOP 封装，数据宽度为 16 位，工作电压 3.3V。

SDRAM 测试程序：此模块的主要功能是先把整个存储器写入数据，再进行读写对比，为了能够充分测试高速存储器的数据正确性，每次写入相同地址数据是不能相同的，每一个地址的数据页也不相同，模块新添加一个计数器用于计算次数，再把地址和写入次数相加，写入相应的存储单元，再读出对比，如果有一个数据错误，设定的某一个信号会发生变化，直到有复位信号归为才变低。另外设定一个信号用于指示测试正在进行，如果测试在进行，则此信号会不断的跳变。

现象：将程序下载到开发板，程序已经绑定了相关的 LED 灯，一个不停的闪烁，表示测试一直在进行中，一个 LED 不亮，表示数据正确，如果数据亮起，表示有错误，复位即可。

第三部分：大板与 FPGA 核心板连接信号汇总

01. 大板输入信号

	信号	标注与情况说明	FPGA PIN	情况说明	备
1	MF 1M	1MHz 的主时钟频率信号	A10		00
2	QD	按下启动按钮 QD 后产生的 QD 脉	J16		
3	PULSE1	中断源信号	G5		
4	PULSE2	中断源信号	L10		
5	CLR/RESET	复位信号	F6		
6	CP1	DZ5 100KHz	H5	被测信号	01
		DZ6 10KHz		被测信号	
7	CP2	DZ7 1KHz	F5	被测信号	10
		DZ8 100Hz		被测信号	
8	CP3	DZ9 10Hz	J6	被测信号	11
		DZ10 1Hz		被测信号	
9	W3		F4		
10	W2		K12		
11	W1		F9		
12	SWA		K11		
13	SWB		F7		
14	SWC		J12		
15	IR7		E12		
16	IR6		J11		
17	IR5		E11		
18	IR4		G12		

02. 频率计实验

信号名称	PIN	备注		信号名称	PIN	备注
CLR	F6			MF	A10	
SWA	K11			SWB	F7	
CP1	H5			CP2	F5	
CP3	J6					
LG8D	A5			LG7D	A7	
LG8C	B5			LG7C	B8	
LG8B	A6			LG7B	A8	
LG8A	B6			LG7A	B10	
LG6D	A9			LG5D	A11	
LG6C	B12			LG5C	C5	
LG6B	A4			LG5B	A12	
LG6A	B14			LG5A	C6	
LG4D	A13			LG3D	T4	
LG4C	C7			LG3C	T5	
LG4B	A14			LG3B	T6	
LG4A	C9			LG3A	T7	
LG2D	T9			LG17	N6	
LG2C	T10			LG16	N8	
LG2B	N4			LG15	N9	
LG2A	N5			LG14	M4	
LG11	M7			LG13	M5	
LG10	M9			LG12	M6	

纯白区域的 PIN 和信号之间是已经通过布线连接好的，不需要通过排线或者#1 线进行连接，绿色区域是需要通过排线或#1 线进行连接的。

需要注意的是，信号在总结的过程中可能会存在写错的问题，如果信号不能被点亮或者数据不同的情况下，请参考各个工程文件中的.ucf 文件。

03. 蜂鸣器实验

信号名称	PIN	备注		信号名称	PIN	备注
Mf	A10			Clr	F6	
Spk	P4					
S8	P5			S4	P9	
S7	P6			S3	G6	
S6	P7			S2	R5	
S5	P8			S1	R7	

纯白区域的 PIN 和信号之间是已经通过布线连接好的，不需要通过排线或者#1 线进行连接，绿色区域是需要通过排线或#1 线进行连接的。

需要注意的是，信号在总结的过程中可能会存在写错的问题，如果信号不能被点亮或者数据不同的情况下，请参考各个工程文件中的.ucf 文件。

04. 交通灯实验

信号名称	PIN	备注		信号名称	PIN	备注
Mf	A10			Clr	F6	
Qd	J16					
TL11	N4			TL5	M5	
TL10	N5			TL4	M6	
TL9	N6			TL3	M7	
TL8	N8			TL2	M9	
TL7	N9			TL1	M10	
TL6	M4			TL0	M11	

纯白区域的 PIN 和信号之间是已经通过布线连接好的，不需要通过排线或者#1 线进行连接，绿色区域是需要通过排线或#1 线进行连接的。

需要注意的是，信号在总结的过程中可能会存在写错的问题，如果信号不能被点亮或者数据不同的情况下，请参考各个工程文件中的.ucf 文件。

05. VGA 实验

信号名称	PIN	备注		信号名称	PIN	备注
Mf	A10			Clr	F6	
Qd	J16					
R	T4			HS	T7	
G	T5			VS	T9	
B	T6					

纯白区域的 PIN 和信号之间是已经通过布线连接好的，不需要通过排线或者#1 线进行连接，绿色区域是需要通过排线或#1 线进行连接的。

需要注意的是，信号在总结的过程中可能会存在写错的问题，如果信号不能被点亮或者数据不同的情况下，请参考各个工程文件中的.ucf 文件。

06. 控制器实验

信号名称	PIN	备注		信号名称	PIN	备注
SWA	K11			W3	N5	
SWB	F7			W2	K12	
SWC	J12			W1	F9	
CLR	F6			SELCTL	B10	
C	B5			ABUS	B8	
Z	A5			SBUS	A8	
IR3	E12			MBUS	A14	
IR2	J11			M	A7	
IR1	E11			S3	C13	
IR0	G12			S2	T15	
T3	C10			S1	H11	
SEL3	A12			S0	R14	
SEL2	C7			MEMW	C8	
SEL1	A11			LAR	E6	
SELO	C6			ARINC	P15	
DWR	A13			LPC	C5	
LIR	D8			PCINC	T14	
PCADD	C11			CIN	R15	
LONG	B6			SHORT	C9	
STOP	C9			LDC	E3	
LDZ	R16					

第四部分：核心板 PIN 汇总

信号名称	PIN	备注		信号名称	PIN	备注
CLK	T8			RESET	L3	

信号名称	PIN	备注		信号名称	PIN	备注
LED4	L12	4 个 LED		KEY4	P11	4 个 KEY 常规 2.75V 按下 0.15V
LED3	L13			KEY3	T12	
LED2	M13			KEY2	R12	
LED1	M14			KEY1	T13	

信号名称	PIN	备注		信号名称	PIN	备注
RXD	D5	UART		SPI_CLK	R11	FLASH
TXD	D6			SPI_DIN	T10	
SCL	N12	EEPROM		SPI_DOUT	P10	
SDA	P12			SPI_CS_N	T3	

信号名称	PIN	备注		信号名称	PIN	备注
SH_CLK	H4	U2_SDRAM		SL_CLK	E10	U3_SDRAM
SH_CKE	C1			SL_CKE	F13	
SH_NCS	M3			SL_NCS	L16	
SH_NWE	L1			SL_NWE	M15	
SH_NCAS	M1			SL_NCAS	M16	
SH_NRAS	M2			SL_NRAS	L14	
SH_DQM0	K2			SL_DQM0	N16	
SH_DQM1	D3			SL_DQM1	E13	
SH_BA0	N1			SL_BA0	K15	
SH_BA1	N3			SL_BA1	K16	
SH_A0	P2			SL_A0	J16	
SH_A1	R1			SL_A1	H14	
SH_A2	R2			SL_A2	H16	
SH_A3	J4			SL_A3	H15	
SH_A4	C2			SL_A4	G16	
SH_A5	B1			SL_A5	F14	
SH_A6	B2			SL_A6	F16	
SH_A7	A2			SL_A7	F15	
SH_A8	C3			SL_A8	K14	
SH_A9	A3			SL_A9	J13	
SH_A10	P1			SL_A10	J14	
SH_A11	B3			SL_A11	G14	
SH_A12	A4			SL_A12	H13	
SH_DB0	G1			SL_DB0	N14	
SH_DB1	H2			SL_DB1	P16	

SH_DB2	H1			SL_DB2	P15	
SH_DB3	H3			SL_DB3	R16	
SH_DB4	J1			SL_DB4	R15	
SH_DB5	J3			SL_DB5	T15	
SH_DB6	K1			SL_DB6	R14	
SH_DB7	K3			SL_DB7	T14	
SH_DB8	D1			SL_DB8	B15	
SH_DB9	E2			SL_DB9	B16	
SH_DB10	E4			SL_DB10	C15	
SH_DB11	E1			SL_DB11	C16	
SH_DB12	F2			SL_DB12	D14	
SH_DB13	F1			SL_DB13	D16	
SH_DB14	F3			SL_DB14	E15	
SH_DB15	G3			SL_DB15	E16	

J1 扩展连接器_40 针：

02	VCC	A5	A6	A7	A8	A9	A10	A11	A12	A13	20
01	GND	B5	B6	B8	B10	B12	B14	C5	C6	C7	19
22	A14	C8	C10	C13	D8	E3	E6	E7	E8	3V3	40
21	C9	T14	R14	T15	R15	R16	P15	P16	N14	GND	39

A10/C9/C10 只能用于时序脉冲，不能做为输出，可以做为输入。

J2 扩展连接器_40 针：

01	VCC	T4	T5	T6	T7	T9	T10	N4	N5	N6	10
11	N8	N9	M4	M5	M6	M7	M9	M10	M11	3V3	20
21	GND	P4	P5	P6	P7	P8	P9	G6	R5	R7	30
31	R9	R12	CLK	L4	L5	L7	L8	K5	K6	GND	40

T10 不能用于 I/O，CLK 为 50M 时序脉冲。

J3 扩展连接器_26 针：

A4	B3	P1	A3	C3	A2	B2	B1	C2	J4	R2	R1	P2	01
H13	G14	J14	J13	K14	F15	F16	F14	G16	H15	H16	H14	J16	02

P2 不能用于 I/O，

J4 扩展连接器_18 针：

GND	D9	F12	H11	G12	J11	J12	K11	K12	H5	J6	L10	M12	01
3V3	C11	D10	D11	D12	E11	E12	F7	F9	F4	F5	F6	G5	02

D10 不能用于 I/O。

J6 扩展连接器_8 针：

01	H5	J6	L10	M12
02	F4	F5	F6	G5